WACV 2027 UPAR Challenge
Fact sheet

This is the fact sheet's template for the WACV 2027 UPAR Challenge. Please fill out the following sections carefully in a scientific writing style. Then, send the compressed project (in .zip format), i.e., the generated PDF, .tex, .bib and any additional files to upar.challenge@gmail.com.

I. Team details

- Team leader name: Syouya Tobita
- Username(s) on Codabench: syouya_tobita
- Team leader affiliation: Independent Researcher (Freelance)
- Team leader email: ayuoysibot8923kite@gmail.com
- Name of other team members (and affiliation): none (individual participation)
- Team website URL (if any): none
- Competition track (mark with X one single option)[1]:
  - ( ) Track 1: Pedestrian Attribute Recognition.
  - (X) Track 2: Attribute-based Person Retrieval.
  - ( ) Track 3: Human Pose Estimation.

This sheet describes our final submission B156C (Codabench submission 1011976), the single submission of the reopened final round. Our entry in the organisers' earlier leaderboard snapshot was B137C (submission 1007281, mADM 0.701395, rank 1 of 49); B156C keeps its three encoders and inference code and changes the DINOv3 head mixture (column-selective), the member weights, the attribute weights of the scorer and the re-ranking (near-duplicate diffusion, attribute-space query expansion, attribute-weighted body stream) (Sec. II-C). We did not submit a paper to the RWS workshop.

II. Contribution details

A. Title of the contribution

Three-model attribute ensemble with spherical multi-resolution heads and gallery-only graph re-ranking.

We fine-tune three generic foundation models (PE-Core-L/14 [1], DINOv3-H+/16 [2], EVA02-L/14 [3]) to predict the 40 UPAR attributes [4]. Within each model family, same-seed recipe soups [5] combine trained parameters without extra inference passes, and separate norm-preserving head soups combine attribute-attention and global evidence. Attribute-dependent member weights produce the gallery probabilities. A query is compared with every gallery image by a weighted Bernoulli scorer, and an automatic re-ranker that uses only the unlabelled gallery combines probability diffusion over visual kNN graphs (with extra diffusion along near-duplicate edges), visual anchors, distance smoothing and an attribute-space expansion of each query from its own top results. There is no cross-query aggregation, no dataset-dependent branching and no manual intervention on test images.

B. Representative image / workflow diagram of the method

Figure 1 shows the inference pipeline described in Sec. II-C.

C. Detailed method description

1) Data and fit boundaries: We use only the organiser-provided Track 2 annotations for Market1501, PA100k and PETA: 97,669 train and 33,407 validation images. The final models are trained on train and validation together (131,075 images after excluding one exact-pixel duplicate), as permitted. No other pedestrian data and no test images are used for fitting; generic pretraining is separate from this task-specific training.

Model selection used leave-one-domain-out predictions: each fold model is trained on two sources and applied to the third. The pooled validation surface has 33,407 images and 3,462 queries. Changes to weights and re-ranking were also checked on a second source-held-out surface built from the train split (28,668 images, 4,325 queries) and on a composition check built only from training images. The last changes were four per-attribute member-weight moves (0.05 moved between the PE and DINOv3 members for upper-body orange and pink, lower-body green and sunglasses) and a re-ranking refinement (diffusion step 1.0 instead of 0.95; attribute-distance weights of Age-Young /1.2, Accessory-Glasses-Sun $\times 1.2$, UpperBody-Color-Red /1.1). Both had to improve the multi-match queries and the overall score on both local surfaces with a train bootstrap interval excluding zero; for the second, the per-cell bar was replaced by a noise-scaled one after our Public history showed that the leaderboard moves with multi-match (MEVID-like) queries. These development surfaces and our Public feedback were reused during selection, so they are not untouched confirmation data.

2) Pretraining, input and architecture: The initialisations are the timm checkpoints vit_pe_core_

[1] If you participated in more than one competition track, you need to share with the organizers one fact sheet per track.

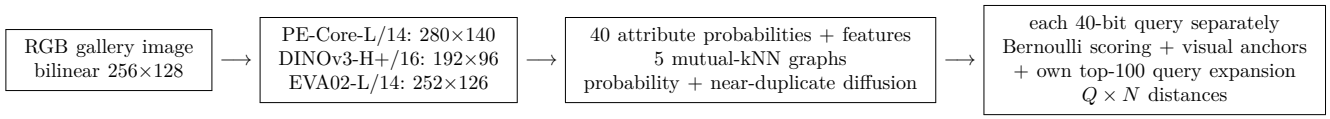


Fig. 1. B156C inference. One forward pass per member and gallery image; no test-time augmentation. All graph statistics use only the unlabelled current gallery; each query is processed without statistics from other queries. Sizes are height×width.

large_patch14_336.fb (image–text contrastive pretraining), vit_huge_plus_patch16_dinov3.lvd1689m (self-supervised, LVD-1689M) and eva02_large_patch14_224.mim_m38m (masked image modelling on Merged-38M). Model cards and the DINOv3 licence are bundled with the model ZIP.

Images are decoded to RGB, resized bilinearly to 256×128, then resized with antialiasing to each member's input and normalised with the backbone's mean and standard deviation. There is no SAHI, sliding window, crop ensemble or flip at inference. PE-Core-L/14 (316.1M parameters, 200 patches) and DINOv3-H+/16 (840.9M parameters, 72 patches) use a linear head plus a 40-query, 256-dimensional attribute-attention residual; EVA02-L/14 (303.2M parameters, 162 patches) uses a linear head. All backbone layers are fine-tuned.

3) Training: There are 13 training recipes, all with seed 20261007: four PE, seven DINOv3 and two EVA recipes. Each uses 400,000 source-balanced samples, effective batch size 128, AdamW (weight decay 0.05), 5% warm-up and cosine decay, head learning rate 10^{-3}, head dropout 0.1 and EMA 0.999 (the EMA weights are used). The first 10% of steps train only the head (LP-FT [6]). Backbone learning rate / layer-wise decay / drop path are $5{\cdot}10^{-5}$/0.85/0.2 for PE and EVA and $4{\cdot}10^{-5}$/0.9/0.3 for DINOv3. Training uses bfloat16 autocast.

The loss is positive-weighted binary cross-entropy. Some recipes add a listwise rank loss with weight 0.1 in which every label vector of the batch acts as a query; selected rank recipes also weight queries by the inverse frequency of their attribute vector. Other variants use per-source masks for labels a source never annotates and for ambiguous colours, or a lower-body part-swap augmentation. At inference the logarithm of the positive class weight is subtracted before the sigmoid (prior correction). The exact per-run configuration files are in the repository (Sec. IV).

4) Parameter and head soups: Backbone soups use weights 0.25 each for the four PE recipes; 0.20/0.30/0.30/0.20 for four DINOv3 recipes trained at 192×96 (base, rank, rank with query balancing, part-swap); and 0.5/0.5 for EVA recipes trained at 224×112 and 252×126 (positional embeddings resampled to the target grid before mixing). For a head weight row with mixture $u = \sum_i a_i w_i$, the spherical head uses $u\,(\sum_i a_i \|w_i\| / \max(\|u\|, 10^{-12}))^{\tau}$. PE head weights are 0.20/0.20/0.30/0.30 (base, rank, mask, part-swap) with $\tau = 1.2$ for query/key, 1.0 for the attention output and 0.6 for the linear base. The DINOv3 head is column-selective: each attribute row mixes six head sources (the four backbone recipes, the label-mask recipe and the mean head of the two 224-pixel recipes) with $\tau = 1$ and the attention output scaled by 1.03. Default rows use 0.18/0.27/0.27/0.18/0/0.10; the rare upper/lower-body colours (Brown, Orange, Pink, Red, Yellow, lower Other) use 0.15/0.225/0.225/0.15/0.15/0.10; the two lower-body types use 0.17/0.255/0.255/0.17/0.15/0; Gender and the two length attributes use 0.20/0.30/0.30/0.20/0/0. EVA uses a 0.5/0.5 head mix with $\tau = 1.2$. Large two-dimensional tensors are stored with per-row int8 quantisation and half-precision scales; the network runs in bfloat16 TorchScript.

5) Scoring and re-ranking: The three probability vectors are mixed per attribute in PE/DINOv3/EVA order: 0.370/0.445/0.185 for age, gender, glasses and hat; 0.375/0.440/0.185 for hair length; 0.305/0.510/0.185 for the two length attributes and bag; 0.320/0.495/0.185 otherwise. All 120 weights are fixed in assets/config.json. The 12 upper-body colour probabilities are pushed halfway towards summing to one and clipped to $[10^{-5}, 1 - 10^{-5}]$. Initial distances combine weighted absolute error and Bernoulli negative log-likelihood against each 40-bit query, with per-image entropy weighting. The 40 attribute weights are cross-fitted attribute reliabilities from early development predictions, multiplied by fixed factors: ×1.15 for the five accessories, ×1.25 for short upper-body length, a further ×1.38 for backpack, ×1.2 for gender and lower-body green, and ÷1.2 for upper-body pink and other and lower-body red and skirt/dress.

Five mutual-kNN graphs use PE, EVA, 0.75PE+0.25EVA, 0.5PE+0.5EVA and 0.5625PE+0.1875EVA+0.25DINOv3 feature similarities (k=20,20,28,28,28; branch weights 0.275,0.230,0.235,0.190,0.070). Edges combine visual similarity, predicted-attribute agreement and closed-neighbourhood Jaccard overlap [7]; more than five confident attribute disagreements remove an edge. Probabilities are diffused [8] for three steps, the graph is re-estimated, and six steps run from the original probabilities, with confidence and member-disagreement weights. Eight further steps then diffuse only along near-duplicate edges, gated by a sigmoid around the 0.80-quantile of the current gallery's mutual-edge cosines in each graph. Base, exact-match and body scoring streams are smoothed on the graphs. Each query's own top results provide visual anchors with gallery hubness normalisation, and head/tail blending combines the streams. Finally, the mean probability vector of the query's own top-100 results is scored as a soft query against the gallery, and its z-scored

distance is added with weight 0.08 times the standard deviation of the query's distances (pseudo-relevance feedback in attribute space). The deployed source b127_jaccard_prob_diffusion.py in the submitted ZIP defines every constant.

6) Use of the scoring feedback (disclosure): The official scoring output of our own submissions shows per-dataset names, query/gallery counts and scores. During the project this feedback informed how we interpreted local proxies (for example, emphasising queries with many correct matches) and a composition check built solely from organiser training images. No hidden images or labels were accessed, the code contains no value read from the test set, and inference does not branch on dataset identity. We disclose this so that the organisers can review it against the rule on hidden-test structure.

D. Challenge results

TABLE I
Codabench results: our final submission B156C (1011976) and, for reference, B137C (1007281) from the organisers' earlier leaderboard snapshot.

Metric	B156C	B137C
mADM	pending	0.701395
mAP	pending	0.418136
Rank-1	pending	0.394486
Rank-5	pending	0.567449
Rank-10	pending	0.642728
mINP	pending	0.293015
mADM MEVID	pending	0.6073
mADM Synth	pending	0.7955

B137C's official run took 258.3 s of the 300 s limit, of which 18.4 s were re-ranking; B156C runs B153C's encoders and re-ranking code (only member weights, three attribute weights and the diffusion step differ), and B153C's re-ranking measured on the official test shape (893 queries, 29,248 images) with identical inputs took within about 1 s of B137C's on the same machine. B156C's own official run (submission 1011976) finished its ingestion in 279.0 s of 300 s, with 21.3 s of re-ranking. A single three-member CUDA plan is required, with canary, finite-value and deadline checks; a failure does not silently switch to a weaker model.

E. Final remarks

Pros. Same-seed recipe soups and low-token inputs fit three large, diverse foundation models into the runtime budget without test-time augmentation. Gallery context helps most for queries with many relevant images, needs no labels and is deterministic. Cons. Resizing to 256×128 loses detail; the development surfaces were reused for selection; the hidden domains differ from the training sources; and the runtime margin on the official hardware is moderate.

III. Additional method details

Please, reply if your challenge entry considered (or not) the following strategies and provide a brief explanation. For each question, mark with X one single option.

- For the competition track associated with this fact sheet, you confirm that you have trained your model on the predefined datasets (X) Yes, ( ) No
  Only the organiser train and validation splits are used for task-specific training.
- Did you use any pre-trained model: (X) Yes, ( ) No.
  If yes, please detail (e.g., model architecture, training strategy, train data, etc.).
  PE-Core-L/14 (image–text contrastive pretraining), DINOv3-H+/16 (self-supervised, LVD-1689M) and EVA02-L/14 (masked image modelling, Merged-38M), all generic public timm checkpoints not trained for pedestrian attributes; all layers are fine-tuned.
- Did you use external data? ( ) Yes, (X) No
  If yes, please detail:
  No external data for task-specific training; the generic pretraining of the checkpoints above is the only external component.
- Did you perform any data augmentation?
  (X) Yes, ( ) No
  If yes, please detail:
  Horizontal flip, random affine (scale, aspect, translation), brightness/contrast jitter; selected recipes also use lower-body part-swap between images of a batch.
- Did you use the provided validation set as part of your training set? (X) Yes, ( ) No
  If yes, please detail:
  The final models are trained on train+validation (131,075 images). Model selection used leave-one-domain-out fold models.
- Did you use any regularization strategies/terms? (X) Yes, ( ) No
  If yes, please detail:
  Weight decay 0.05, head dropout 0.1, drop path 0.2–0.3, layer-wise learning-rate decay, LP-FT, EMA 0.999, per-source label masks and parameter soups.
- Did you use any pre-processing steps? (X) Yes, ( ) No
  If yes, please detail:
  RGB decoding, bilinear resize to 256×128, antialiased resize to each member's input size and per-backbone normalisation. No crops or flips at test time.
- Did you use any post-processing steps? (X) Yes, ( ) No
  If yes, please detail:
  Prior correction of the logits, per-attribute member weighting, partial renormalisation of the upper-

body colour group, probability clipping, entropy- and attribute-weighted Bernoulli scoring and the automatic gallery-only re-ranking of Sec. II-C, including near-duplicate diffusion and the attribute-space expansion of each query from its own top-100 results (per query, no cross-query statistics, no labels).

- Did you use Vision Language Models (VLMs) of any kind? (X) Yes, ( ) No
  If yes, please detail:
  Only as an image backbone: PE-Core-L/14 is the vision encoder of Meta's Perception Encoder, pre-trained contrastively on image–text pairs. We fine-tune its image tower only; no text encoder, prompts or language model are used in training or inference.

IV. Code repository

Link to a code repository with complete and detailed instructions so that the results obtained on Codabench can be reproduced locally. This includes a list of requirements, pre-trained models, and so on. Note, training code with instructions is also required. This is required for all participants to appear in the final ranking. Organizers strongly encourage the use of docker to facilitate reproducibility.

Code repository: REPOSITORY_URL

Trained weights: WEIGHTS_URL (the exact submitted file B156C_B155C_K2_RERANK.zip, 1,333,500,417 bytes), SHA-256:
1279da344c73aa8c67ee55703ba7e2c6e7c1b9607e3a182e9c9e70499fb30844

The repository provides an inference Docker image built on the Codabench runner image (pytorch/pytorch: 2.4.1-cuda12.1-cudnn9-runtime) that verifies and runs the submitted ZIP through its run.py on a public split and scores the output, a training Docker image with the recorded environment (PyTorch 2.8, timm 1.0.20) and a script that retrains the 13 recipes, the full source code, the original configurations, the soup/head/packaging scripts and an offline verification script for all delivered files.

References

[1] D. Bolya et al., "Perception encoder: The best visual embeddings are not at the output of the network," arXiv preprint arXiv:2504.13181, 2025.

[2] O. Siméoni et al., "DINOv3," arXiv preprint arXiv:2508.10104, 2025.

[3] Y. Fang, Q. Sun, X. Wang, T. Huang, X. Wang, and Y. Cao, "EVA-02: A visual representation for neon genesis," arXiv preprint arXiv:2303.11331, 2023.

[4] A. Specker, M. Cormier, and J. Beyerer, "Upar: Unified pedestrian attribute recognition and person retrieval," in Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), January 2023, pp. 981–990.

[5] M. Wortsman, G. Ilharco, S. Y. Gadre, R. Roelofs, R. Gontijo-Lopes, A. S. Morcos, H. Namkoong, A. Farhadi, Y. Carmon, S. Kornblith, and L. Schmidt, "Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time," in International Conference on Machine Learning (ICML), 2022.

[6] A. Kumar, A. Raghunathan, R. Jones, T. Ma, and P. Liang, "Fine-tuning can distort pretrained features and underperform out-of-distribution," in International Conference on Learning Representations (ICLR), 2022.

[7] Z. Zhong, L. Zheng, D. Cao, and S. Li, "Re-ranking person re-identification with k-reciprocal encoding," in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.

[8] D. Zhou, O. Bousquet, T. N. Lal, J. Weston, and B. Schölkopf, "Learning with local and global consistency," in Advances in Neural Information Processing Systems (NeurIPS), 2004.